SE AI Agent Index: A Dataset of Software Engineering Practices for Agentic AI Systems

Mustahid Hasan
William & Mary
Williamsburg, Virginia, USA
mhasan02@wm.edu

Antu Saha
William & Mary
Williamsburg, Virginia, USA
asaha02@wm.edu

Oscar Chaparro
William & Mary
Williamsburg, Virginia, USA
oscarch@wm.edu

***Abstract*—Agentic AI systems have transformed software development through autonomous task execution across code generation, browser automation, and enterprise workflows. As organizations deploy these agents into production, the Software Engineering (SE) practices behind their development workflow become critical knowledge for both research and industry. However, this knowledge remains scattered across engineering blogs, conference talks, technical documentation, and issue trackers, which makes it difficult for practitioners to understand how teams design, test, and maintain agentic systems at scale. To address this gap, we present the *SE AI Agent Index*, a structured dataset that documents internal Software Engineering (SE) practices for 13 AI agents across eight core practice areas, with coverage of test strategies, deployment pipelines, code review process, and issue resolution strategy. Our semi-automated technique combines LLM-assisted search, LLM-judge via system prompt, and two-stage manual quality validation, where every claim links to exact evidence from publicly accessible sources for full traceability. We found that the companies of these agents adopt their own agents in daily development, emphasising additional efforts on observability and trajectory rather than conventional tests and distinct deployment trends between enterprise and open-source agents. Moreover, our case studies on 2 agentic systems demonstrate how the dataset can be utilized to extract key insights on agent development, pitfalls, and internal tools.**

I. Introduction

Artificial Intelligence (AI) agents—autonomous programs that plan multi-step workflows, invoke tools, and act on behalf of users with minimal human involvement—have rapidly moved from research prototypes to production deployments [1–3]. Agentic AI has expanded across domains from scientific research and healthcare to finance, legal services, and enterprise automation, fundamentally altering how complex work is structured and executed across sectors [1]. Software engineering is among the domains where this shift is most pronounced: agents are now integrated throughout the SDLC, automating tasks such as code generation and program repair [4]. Because of the autonomy of agentic systems and the non-deterministic, data-driven nature of Large Language Models (LLMs), the development process of building these agents is likely different from the non-agentic traditional systems [5, 6]. As these systems grow in capability and adoption, their engineering teams must continuously evolve how they develop, test, and maintain these systems—yet these software engineering practices remain unclear to the research community [7].

Existing efforts to document AI systems and agents, including model cards [8], system cards [9], and the Foundation Model Transparency Index [10], focus on *what* systems can do, their societal impact, and decision-making transparency.

The AI Agent Index [7] and the 2025 AI Agent Index [11] documented agents by selecting publicly accessible systems and extracting evidence across capabilities, safety, and governance dimensions. While these resources are valuable for understanding agent behavior, they say nothing about the software engineering practices of the teams who build them, including the testing methodologies, CI/CD pipelines, code review processes, bug-solving workflows, or documentation culture they follow in practice. This gap has important consequences on two ways: in industry, organizations building new agentic systems have no structured reference for understanding which development patterns, tools, or maintenance strategies established teams rely on ; in academia, the absence of such a baseline leaves open a rich but underexplored research direction: how the software engineering practices behind these rapidly evolving systems are themselves changing.

To fill this gap, we introduce the ***SE AI Agent Index***: the first structured dataset investigating the software engineering practices of AI agent development teams to study the SE practices of these agentic systems.

Using open coding methodology [12], we annotate and analyze agents grounded in evidence from public repositories, engineering blogs, and technical talks.

This paper makes the following **Contributions** :

- ***SE AI Agent Index***: Structured SE practices for 13 AI agent systems, generated from the publicly accessible websites, repositories and subsequently reviewed and validated by researchers across eight SE practice dimensions
- **Semi-automated Indexing Framework**: A structured framework that automatically generates SE index, screens each entry via an LLM-as-Judge method [13] across five semantic dimensions, and requires researchers to manually validate qualifying data against the same five criteria to cross-check LLM judgments. [1]
- **Pattern Analysis and Case Studies**: A cross-agent analysis of SE patterns across all eight dimensions, with two in-depth case studies to understand the trends of these agentic systems.

[1]Available at: [https://github.com/musta55/se-ai-agent-index.git]

II. Related Work

The AI Agent Index [7] documented technical components and safety features of deployed agents but focused on product-level transparency rather than the internal SE practices of the teams building them. Wang *et al.* [14] empirically studied developer experiences across LLM-based agent frameworks, identifying SDLC challenges and comparing frameworks on efficiency and maintainability, though their scope remains at the framework level. Alenezi [15] argues that SE must reorganize around orchestration, verification, and human-AI collaboration as agentic systems proliferate. Individual practice areas have also attracted focused study: Hasan et al. [16] examined testing practices in open-source agents. While these works address individual practice areas or framework-level concerns, none documents SE practices across multiple deployed agents and practice areas.

III. Dataset Description and Access

The dataset contains a collection of software engineering practices of 13 agentic systems. This section discusses the structure of our dataset.

A. Schema Definition

Each agent record follows a consistent schema: practices are organized into 7 software engineering *dimensions*, covering:

- **SE Methodology** — development methodology (*e.g.*, Agile, Scrum), sprint structure, team size, and planning practices.
- **Code Review Process** — pull request policies, approval requirements, and automated quality checks.
- **Testing** — test frameworks, coverage targets, evaluation strategies, and red-teaming practices.
- **CI/CD Pipeline** — deployment process, release cadence, and rollback procedures.
- **Bug Solving** — issue triage workflows, debugging approaches, and recurring engineering problem patterns.
- **Team Communication** — internal communication channels, design documentation, decision-making processes, and contributor onboarding.
- **Holistic Documentation** — documentation culture, tooling, maintenance standards, and ownership models.

Within each *dimension*, information is captured through four *sub-dimensions*: *Practices* (workflows and methodologies the team follows), *Tools* (specific tools, frameworks, and services used), *Stakeholders* (who are involved and in what role), and *Others* (additional context not covered by the preceding three). Each sub-dimension contains multiple captured information with unique query identifiers (*q_id*), which we call them 'entries.'

Each 'entry' in our dataset (i.e., spreadsheet) is backed with seperate inventory stored as JSON, which contains direct quotes drawn from the cited sources. To support verification and accessibility, we developed a retrieval tool that allows to look up the details of each entry by its *q_id*. Given a *q_id*, the tool returns the corresponding *Summary* along with an evidence array of direct quotes drawn from the cited sources. Each citation further records when the source was published (if available) and when it was accessed (*Date_Pulled*). This allows anyone to verify that the entry in our dataset faithfully represents its cited source.

B. Dataset Statistics

As of the index cutoff date (May 18, 2026), the dataset spans 13 agents, indexed across 7 SE practice dimensions to yield 91 subdimensions and over 364 structured entries. The 260 supporting sources (range: 8–43 per agent, mean: 20.0) originate from three broad origin types: official engineering blogs and documentation (44%), GitHub repositories and issue trackers (24%), and external sources—technical papers, security reports, podcasts, and practitioner interviews (12%).

C. Storage and Access

The dataset is available in a spreadsheet where we provide a curated summary of the agents in 7 dimensions.

Also, to support transparency and reproducibility, we developed a retrieval tool that allows anyone to look up any indexed entry by its *q_id*. Given a *q_id*, the tool returns the corresponding *Summary* along with an evidence array of direct quotes drawn from the cited sources. Each citation further records when the source was published (if available) and when it was accessed (*Date_Pulled*). This top-down approach makes the process of indexing robust and comprehensive. To review the indexing by the LLM-as-judge, the user can use a script to review the agent's quality and checklist.

IV. Methodology for Dataset Creation

The goal of the *SE AI Agent Index* dataset is to understand how agentic systems are developed, maintained, and deployed in real-world scenarios. The dataset creation technique consists of AI agent selection, data extraction process, and semi-automated data quality validation as shown in Fig 1.

A. AI Agent Selection

We constructed our AI agent pool from the 30 agents documented in the 2025 AI Agent Index [7], which systematically screened 95 candidate agents using three criteria: *Agency* (autonomous, goal-directed behavior with broad applicability), *Impact* (demonstrated public interest, market presence, or developer adoption), and *Practicality* (publicly accessible and deployable in real-world settings). By leveraging this curated benchmark, we grounded our dataset on agents that are already recognized as representative and influential examples of modern agentic systems.

These 30 agents span three major categories: chat-based agents, browser-based agents, and enterprise workflow platforms. To ensure diversity across application domains, we randomly selected 4–5 agents from each category, resulting in a total of 13 agents. Our final dataset includes: (i) *chat-based agents* (Anthropic Claude Code, Amazon Q Developer, Google Gemini CLI, Google Gemini); (ii) *browser-based agents* (Browser Use, Opera Neon, Kimi Ok Computer, Perplexity Comet, Alibaba Mobile Agent); and (iii) *enterprise workflow platforms* (Google Gemini Enterprise, Glean Agents, IBM watsonx Orchestrate, HubSpot Breeze Studio).

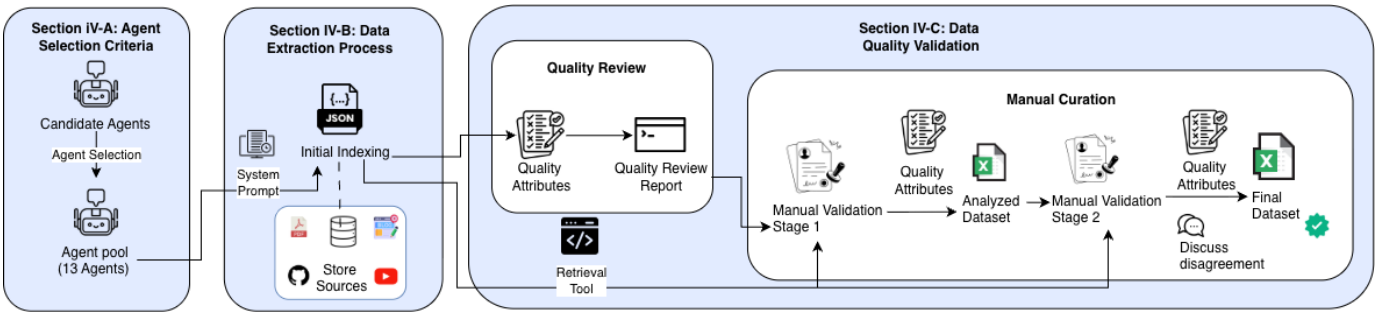


Fig. 1: Overview of the methodology for creating the *SE AI Agent Index* Dataset

B. Data Collection

For the selected 13 AI agents, our goal is to collect *SE practice entries* according to our dataset schema, *i.e.*, seven dimensions (*e.g.*, Testing Practices, CI/CD Pipeline, *etc.*) and four sub-dimensions (*e.g.*, practices, tools, *etc.*) (see Section III-A). To do so, we analyzed the publicly available resources for each agent by leveraging Claude Sonnet 4.6 [17] via a carefully designed system prompt Figure 2.

The system prompt specifically instructs the LLM to search public sources (*e.g.*, repositories, developer blogs, official documentation, conference talks, technical papers, and issue trackers) to extract relevant information (*i.e.*, specific quotes from the resource with reference) for each entry in our dataset schema (see Section III-A). Moreover, it also instructs the LLM to generate a clear and concise summary utilizing the extracted relevant information, which serves as the initial entry in our dataset.

To design the system prompt, we followed an iterative approach. We started with the high-level overview of the goals, introduced the 7 dimensions III (*e.g.*,testing, SE methodology, code review process..) to generate indexing for an agent. After validating the response, we refined the prompt by specifying each dimension in detail, adding guidelines to distinguish between product features and internal development practices with examples, and introducing constraints to avoid documenting unnecessary information. 2 authors discussed about the responses generated by the prompt and validate the prompt based on response. Through continuous refactoring, we finalized our system prompt. In parallel, we evaluated three LLMs for generating the initial indexing (*i.e.*, JSON) using 3 LLM model (Gemini 2.5 pro, Gemini 3.1 pro, Claude Sonnet 4.6). We selected Claude Sonnet 4.6 for our indexing task, as it produced the most reliable and consistent results throughout the experiment. To ensure prompt quality, two authors independently reviewed the responses generated by each candidate model and used these observations to further validate and refine the prompt. Through continuous refactoring, we finalized the system prompt.

Through this process, we construct an initial index of 13 JSON files (one per agent), which are then forwarded to the quality validation pipeline detailed in Section IV-C.

Query

You are an expert SE Research Agent. Your goal is to analyze how {*agent_name*} is developed, maintained, and deployed by its engineering team and build a standardized SE Index.

Instructions

Search public sources (repositories, agent development blogs, documentation, conference talks, papers, and issue trackers) and extract evidence across **7 SE practice sections**, each broken into *Practices*, *Tools*, *Stakeholders*, and *Others*.

Output Format (JSON)

```
{ "Agent_Name": "...",
  "SE_Practices": {
    "<Section>": {
      "Practices": [{
        "q_id": "q_N",
        "Summary": "...",
        "evidence": [{
          "quote": "<verbatim text>",
          "citation": "[N]" }] }] },
  "References": {
    "[N]": { "Website_Type": "...",
             "Website_Link": "...",
             "Date_Created": "...",
             "Date_Pulled":  "..." }}}
```

Fig. 2: Structure of the SE indexing system prompt used to generate a JSON index per agent.

C. Data Validation

Since LLM-generated initial entries for our dataset schema may contain noise or inconsistencies, we applied a semi-automated validation process to ensure the quality and reliability of the final entries and their associated information sources. The goal of this process is to produce high-quality entries for each software engineering practice captured in our dataset. Our validation pipeline consists of two stages: (i) automated validation, where we use an LLM to check and flag potential issues in the dataset, and (ii) a two-stage manual validation, where two paper authors inspect and refine the entries to ensure accuracy and consistency.

Both the automated and manual validation stages evaluate the dataset using five core semantic quality attributes: (1) *content focus*, which ensures that the extracted content documents internal development practices rather than product features; (2) *anti-hallucination*, which verifies that all claims can be traced back to their cited sources; (3) *quote correctness*, which confirms that quotes are copied verbatim from publicly available sources instead of being paraphrased; (4) *evidence-summary alignment*, which validates that each quoted evidence directly supports its associated summary; and (5) *unavailability check*, which ensures that items marked as "not available" are indeed unavailable in the source materials.

Automated Validation. As the first stage of our validation pipeline, we employ an LLM-as-judge framework [13] to automatically assess the quality of the indexed entries. Specifically, we use Claude Sonnet 4.6 [17] to evaluate each entry across the five semantic quality dimensions described above. For each dimension, the model assigns one of three labels: 'Fail', indicating that a claim cannot be traced to the cited evidence or that the source link is inaccessible; 'Warn', indicating partial evidence support or that the entry focuses on product features instead of internal engineering practices; and 'Pass', indicating no detected issues. We use these assessments to prioritize entries during manual validation, particularly those flagged with warnings or failures. Nevertheless, all generated entries from the initial indexing stage are manually inspected regardless of their automated assessment outcome.

Manual Validation. We conduct a two-stage manual validation process to ensure that the final dataset contains only publicly accessible and directly verifiable information. In both stages, we use the retrieval tool described in Section III-A to access entries by their *q_id* and review the associated summaries, quotes, citations, and automated quality reports in a unified interface.

In the first stage, one researcher systematically reviews each entry against the original web sources. This process includes verifying that quoted evidence is copied verbatim from publicly available documents, confirming that summaries are supported by the cited evidence, and validating compliance with the five semantic quality dimensions. During this stage, we remove unsupported or unverifiable claims, revise inaccurate summaries, and augment entries with additional evidence when relevant information is publicly available but initially missing.

In the second stage, a second researcher reviews the curated entries using the same validation criteria and retrieval interface. The reviewer annotates entries with suggestions for additions, removals, or corrections based on the supporting evidence. Finally, both researchers discuss and resolve all disagreements to produce the finalized version of the dataset.

V. Usage Scenarios and Research Implications

The *SE AI Agent Index* enables both practical and research-oriented analysis of how modern AI agents are engineered in real-world settings. By systematically documenting development workflows, testing strategies, deployment practices, and maintenance processes across different agentic systems, the dataset provides a foundation for studying how software engineering practices are evolving in response to autonomous and LLM-driven systems. In this section, we discuss several usage scenarios and research implications enabled by the dataset.

A. Benchmarking Engineering Practices for AI Agents

The *SE AI Agent Index* provides the first structured reference for understanding how production-grade AI agents are engineered, tested, deployed, and maintained in practice. Since engineering knowledge for agentic systems is currently scattered across blogs, repositories, issue trackers, and technical talks, organizations building new agents often lack concrete guidance on reliable development workflows .Our dataset consolidates these practices into a unified schema, enabling direct comparison across testing strategies, deployment workflows, review practices, and debugging approaches.

The dataset already captures recurring infrastructure and engineering patterns across agents, including the use of Kubernetes in HubSpot Breeze Studio, OpenTelemetry in IBM watsonx Orchestrate, GitHub Actions and Docker-based testing in Gemini CLI, and LLM-based evaluation workflows in Google Gemini Enterprise and Amazon Q Developer. These indexed practices can support automated information extraction research and enable systems that answer questions such as "Which agents use LLM-as-judge evaluation?" or "What testing practices reduce flaky agent behavior?" From an industrial perspective, organizations building new agents can use the dataset as a reference for designing testing pipelines, CI/CD workflows, code review policies, and issue-management strategies.

VI. Limitations, Threats to Validity

A. Limitations

The dataset reflects practices documented through May 18, 2026. Agents updating practices post-cutoff require reindexing, which entails re-scraping documentation, re-running the automated pipeline, and conducting manual validation. While automated extraction is computationally inexpensive, manual validation requires significant time and human effort. To ensure reproducibility and justify indexing correctness, we store the exact website versions and pull timestamps.

Additionally, some agent-specific practices lack explicit documentation. To mitigate the risk of missing valuable sources, we systematically target official documentation, primary GitHub repositories, and developer guides. We also manually augment the dataset with exact quotes missed during initial indexing to maximize transparency and comprehensiveness.

For closed-source agents, we infer practices from related public repositories, such as CLI tools or SDKs. Although the extraction model (Claude 4.6 Sonnet) is susceptible to source-selection bias, manual verification confirmed it consistently chose the correct sources. Finally, even with automated validation and structured schemas, human interpretation of SE practices remains subjective.

B. Threats to Validity

Internal validity. Organizations may not document development practices explicitly for individual agents. When agent-specific documentation is unavailable, we infer practices from organization-wide engineering principles and methodologies, such as a company's publicly documented adoption of trunk-based development, continuous deployment pipelines, or model-based evaluation frameworks, published in engineering blogs or official technical documentation. For example, if an organization documents using LLM-as-judge evaluation broadly across its products, we treat that practice as applicable to their agent unless contradicted by agent-specific evidence. This inference may introduce inaccuracy when an agent team deviates from broader organizational practices in ways not captured by public documentation.

External validity. Our case study documents SE practices for 13 prominent AI agents and does not claim these findings generalize to all agents in the ecosystem. The more relevant external validity concern is whether the documented practices generalize to the wider population of *teams* building and maintaining AI agents. The 13 agents studied are primarily well-resourced, publicly documented systems from organizations with active developer communities; teams building specialized, proprietary, or resource-constrained agents may adopt substantially different practices. Researchers should interpret our findings as characterizing a representative but non-exhaustive cross-section of current agent development, rather than universal practice across the field.

VII. Conclusion

We present a structured dataset documenting software engineering practices across 13 AI agents through systematic extraction and manual verification. The dataset addresses the gap in aggregated, evidence-based documentation of how teams build, test, deploy, and maintain agentic systems. Our methodology combines LLM-assisted search with rigorous human curation using evidence from official sources. The dataset serves researchers studying SE practice evolution in AI systems and organizations building new agents. The documented methodology enables extension to additional agents or deeper analysis of specific SDLC areas, to support empirical studies of how agent development practices mature.

References

[1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[2] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Lin, T. Gui, Y. Qu, X. Qiu, X. Huang, and Z. Wei, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[4] A. Soni, A. Kumar, R. Arora, and R. Garine, "Integrating ai into the software development life cycle: best practices, tools, and impact analysis," *Tools, and Impact Analysis (June 10, 2023)*, 2023.

[5] A. E. Hassan, H. Li, D. Lin, B. Adams, T.-H. Chen, Y. Kashiwa, and D. Qiu, "Agentic software engineering: Foundational pillars and a research roadmap," *arXiv preprint arXiv:2509.06216*, 2025.

[6] F. Fournier, L. Limonad, and Y. David, "Agentic ai process observability: Discovering behavioral variability," 2025. [Online]. Available: https://arxiv.org/abs/2505.20127

[7] S. Casper, L. Bailey, R. Hunter, C. Ezell, E. Cabalé, M. Gerovitch, S. Slocum, K. Wei, N. Jurkovic, A. Khan, P. Christoffersen, A. P. Ozisik, R. Trivedi, D. Hadfield-Menell, and N. Kolt, "The AI agent index," *arXiv preprint arXiv:2502.01635*, 2025.

[8] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," in *Proc. Conf. Fairness, Accountability, and Transparency (FAccT)*, 2019, pp. 220–229.

[9] F. Gursoy and I. A. Kakadiaris, "System cards for AI-based decision-making for public policy," *arXiv preprint arXiv:2203.04754*, 2022.

[10] R. Bommasani, K. Klyman, S. Kapoor, N. Maslej, B. Xiong, D. Zhang, and P. Liang, "The foundation model transparency index," *arXiv preprint arXiv:2310.12941*, 2023.

[11] L. Staufer, K. Feng, K. Wei, L. Bailey, Y. Duan, M. Yang, A. P. Ozisik, S. Casper, and N. Kolt, "The 2025 AI agent index: Documenting technical and safety features of deployed agentic AI systems," *arXiv preprint*, 2025, available: https://aiagentindex.mit.edu.

[12] A. Strauss and J. Corbin, *Basics of Qualitative Research: Grounded Theory Procedures and Techniques*. Newbury Park, CA: Sage Publications, 1990.

[13] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[14] Y. Wang, X. Xu, J. Chen, T. Bi, W. Gu, and Z. Zheng, "An empirical study of agent developer practices in AI agent frameworks," *arXiv preprint*, 2025.

[15] M. Alenezi, "Rethinking software engineering for agentic AI systems," *arXiv preprint arXiv:2604.10599*, 2026.

[16] M. M. Hasan, H. Li, E. Fallahzadeh *et al.*, "An empirical study of testing practices in open source AI agent frameworks and agentic applications," *Empirical Software Engineering*, 2026.

[17] Anthropic, "Claude sonnet 4.6," https://www.anthropic.com/claude/sonnet, 2025, accessed: May 2026.